Code Readability

Code = for humans

Compiled code = for computers

Natural Language conventions work well for humans

In particular: saying something in a verbose/unusual way carries implied meaning

Ex: you'll vs you will

x = x + 1 vs x++

Java Conventions: Collections

Java Collections:

- List/ArrayList
- Set/HashSet
- Map/HashMap

[] arrays are **unusual** in practice

- typically only if you're doing memory management performance optimizations

Java Conventions: Iteration

Iteration over a List/Set/Collection/Iterable:

Old:

```
List<String> list = new ArrayList<>();
for (int i = 0; i < list.size(); i++) {
    String s = list.get(i);
    System.out.println(s);
}
```

Java Conventions: Iteration

Iteration over a List/Set/Collection/Iterable:

New:

```
List<String> list = new ArrayList<>();
for (String s : list) {
    System.out.println(s);
}
```

Java Conventions: Function Pointers

Instead of parameterizing a **type**, allows parameterizing **behavior**

Ex: sorting objects needs a Comparator

How to pass this to a method?

- Everything is an Object
- Create an interface with the necessary method signature
- Pass an implementation of the interface

Java Conventions: Function Pointers

Problem: Creating a whole class just to sort one list is clunky

Solution:

Anonymous inner classes

```
List<String> strings = new ArrayList<String>();
Collections.sort(strings, new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        // reverse the standard order
        return -1*s1.compareTo(s2);
    }
});
```

Java Conventions: Function Pointers

Problem: Anonymous inner classes are clunky

Solution: Lambdas (introduced in Java 8)

```
List<String> strings = new ArrayList<String>();
Collections.sort(strings, (s1, s2) -> -1*s1.compareTo(s2));
```

Java Conventions: Function Pointers

Problem: Lambdas remove **too much** info

Solution: More than 1 lambda in a scope → create named, typed variables for them

```
List<String> strings = new ArrayList<String>();
Comparator<String> reverseSort = (s1, s2) -> -1*s1.compareTo(s2);
Collections.sort(strings, reverseSort);
```

Lambdas as Constants

Combined with static constants on interfaces, you can easily and compactly return values without losing API flexibility.

```
public interface ComputeResult {
    static ComputeResult SUCCESS = () -> ComputeResultStatus.SUCCESS;
    static ComputeResult FAILURE = () -> ComputeResultStatus.FAILURE;

    ComputeResultStatus getStatus();

    public static enum ComputeResultStatus {
        SUCCESS,
        FAILURE;
    }
}
```

Java Conventions: Fluent Interfaces

Often combined with lambdas

- Allows **chaining** method calls, by calling a method on the returned object

Without fluent interface:

```
public static void main(String[] args) {
   Widget w = new Widget();
   w.addGearToWidget();
   w.addCrankToWidget();
}

public static class Widget {
    public void addGearToWidget() {/* do things */}
    public void addCrankToWidget() {/* do things */}
}
```

Java Conventions: Fluent Interfaces

Often combined with lambdas

- Allows **chaining** method calls, by calling a method on the returned object

With fluent interface:

```
public static void main(String[] args) {
   Widget w = new Widget()
            .addGearToWidget()
            .addCrankToWidget();
}

public static class Widget {
    public Widget addGearToWidget() {/* do things */ return this;}
    public Widget addCrankToWidget() {/* do things */ return this;}
}
```

Design Patterns

Many problems have a common structure

Design patterns are like a toolbox to help solve common problems

For each pattern:

- Know what type of problem to apply it to
- Know how to adapt the pattern to the problem
- Recognize complex uses of the pattern

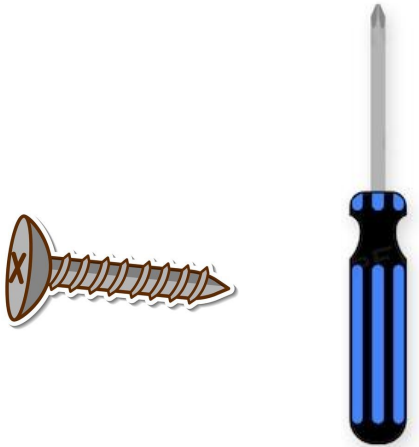
We'll be covering some common ones, there are many more (https://refactoring.guru/design-patterns has a good list)

Design Patterns

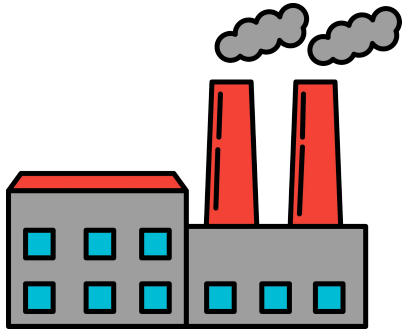
When should you use a design pattern?

Usually, when you have no other choice

Tools, not building blocks - don't go looking for ways to use them, let the problems come to you



Design Patterns



Builder Pattern (related: Factory pattern)

Problem: Some object that will be widely passed around can't easily be constructed all at once with a constructor.

How to avoid having mutable state all over the codebase?

```
public class Widget {

    private int value1;
    private int value2;

    public int getValue1() {
        return value1;
    }

    public int getValue2() {
        return value2;
    }

    /*
     * These setters aren't great - they let anything in the entire codebase
     * muck around with the values of a Widget. This encourages code that's very
     * interconnected and hard to reason about (especially with thread safety)
     */
    public void setValue1(int value1) {
        this.value1 = value1;
    }

    public void setValue2(int value2) {
        this.value2 = value2;
    }
}
```

Design Patterns

Builder Pattern (related: Factory pattern)

Solution: Create a helper class that handles construction; the resulting object is **immutable**, while the helper class (WidgetBuilder/WidgetFactory) contains the mutable data

Note the **return values** - this is a **fluent** interface, meaning the code that uses it has easily-readable chained calls:

```
new WidgetBuilder()
.setValue1(1)
.setValue2(2)
.build()
```

```
public class Widget {
    private final int value1;
    private final int value2;

    public Widget(int value1, int value2) {
        this.value1 = value1;
        this.value2 = value2;
    }

    public int getValue1() {
        return value1;
    }

    public int getValue2() {
        return value2;
    }
}
public class WidgetBuilder {
    private int value1;
    private int value2;

    public WidgetBuilder setValue1(int value1) {
        this.value1 = value1;
        return this;
    }

    public WidgetBuilder setValue2(int value2) {
        this.value2 = value2;
        return this;
    }

    public Widget build() {
        return new Widget(value1, value2);
    }
}
```

Design Patterns

Builder Pattern (related: Factory pattern)

Fancy Solution: Make the helper class a static inner class

Caller:

```
Widget.builder()
.setValue1(1)
.setValue2(2)
.build()
```

Extra bonus: with overloaded constructors, there's no way to make both value1 and value2 independently options, since they're both ints. With builders, you can!

```
public class Widget {

    private Widget(int value1, int value2) {
        this.value1 = value1;
        this.value2 = value2;
    }

    public static Builder builder() {
        return new Builder();
    }

    public static class Builder {

        private int value1;
        private int value2;

        private Builder() {
            // only grab through Widget.builder()
        }

        public Builder setValue1(int value1) {
            this.value1 = value1;
            return this;
        }

        public Builder setValue2(int value2) {
            this.value2 = value2;
            return this;
        }

        public Widget build() {
            return new Widget(value1, value2);
        }
    }
```

Your Turn!

1. Follow the link from Brightspace to check out the CodingExercises repo; use `./gradlew eclipse` to set up a project to import into your IDE
   a. If you need to update your JUnit version, do that before running `gradlew eclipse`
2. In exercise1, use the Builder pattern to fix the TODO in Train.java; when you're done, you should have two different classes, and the TrainTest should no longer compile

Design Patterns

```
private final Database database;
private final NetworkedDrive remoteFileSystem;

public Widget() {
    database = connectToDb();
    remoteFileSystem =  scanAndMountFileSystem();
}
```

Dependency Injection

Problem: Component1 depends on Component2 and Component3, and is set up to connect to/create them in the constructor.

How do I unit test this code?

Solution: Force the constructor to take in an already-build Component2 and Component3, and move the connection/creation logic elsewhere

Fun fact: Using TDD (test-driven development) forces you to set everything up this way from the beginning

```
private final Database database;
private final NetworkedDrive remoteFileSystem;

public Widget() {
    this(connectToDb(), scanAndMountFileSystem());
}

public Widget(Database db, NetworkedDrive drive) {
    this.database = db;
    this.remoteFileSystem = drive;
}
```

Design Patterns

Dependency Injection (part 2)

Problem: 3 different components all use the same (expensive-to-build) sub-component.

How do I get them to share an instance?

Solution:

Use a dependency injection library: Guice, Dagger

Good to recognize, since codebases using an annotation framework never explicitly call a constructor, which can be confusing to see

```
public class WidgetModule {

    @Provides
    @Singleton
    Database provideDatabase() {
        return connectToDb();
    }

    @Provides
    Widget provideWidget(Database database, NetworkedDrive drive) {
        return new Widget(database, drive);
    }
```

```
public class Widget {

    private final Database database;
    private final NetworkedDrive remoteFileSystem;

    @Inject
    public Widget(Database db, NetworkedDrive drive) {
        this.database = db;
        this.remoteFileSystem = drive;
    }
```

Design Patterns

Lazy Initialization

Problem: Some object relies on an expensive computation to be able to work properly

How to prevent this slowing down server/process/component startup?

```
public class Widget {

    private final Database database;

    public Widget(Database database) {
        this.database = database;
        database.initializeSchema();
    }

    public void doAThing() {
        // nothing for now
    }
```

Design Patterns

Lazy Initialization

Solution: initialize on demand

Bonus: May never need to initialize

This is fragile, so only use if needed, and make sure to test!

Ex: what happens if initializeSchema() fails?

```
public class Widget {

    private final Database database;

    // Default boolean value is false, but
    // it's good to be clear
    private boolean initialized = false;

    public Widget(Database database) {
        this.database = database;
    }

    public void doAThing() {
        initializeIfNecessary();
        actuallyDoThing();
    }

    private void actuallyDoThing() {
        // nothing for now
    }

    private void initializeIfNecessary() {
        if (!initialized) {
            database.initializeSchema();
            initialized = true;
        }
    }
```

Design Patterns

Wrapper/Decorator

Problem: I have a complicated object, and I want to add a small amount of functionality to it

How can I easily augment every method of a class?

```
public class FancyMap<K, V> {

    private final Map<K, V> wrappedMap;

    public FancyMap(Map<K, V> wrappedMap) {
        this.wrappedMap = wrappedMap;
    }

    public V put(K key, V value) {
        prettyPrintToLogFile("putKey", key);
        prettyPrintToLogFile("putValue", value);
        return wrappedMap.put(key, value);
    }

    public V get(K key) {
        prettyPrintToLogFile("getKey", key);
        return wrappedMap.get(key);
    }

    public int size() {
        return wrappedMap.size();
    }
```

Design Patterns

Wrapper/Decorator

Why not just extend a class?

```
public class FancyMap<K, V> {

    private final Map<K, V> wrappedMap;

    public FancyMap(Map<K, V> wrappedMap) {
        this.wrappedMap = wrappedMap;
    }

    public V put(K key, V value) {
        prettyPrintToLogFile("putKey", key);
        prettyPrintToLogFile("putValue", value);
        return wrappedMap.put(key, value);
    }

    public V get(K key) {
        prettyPrintToLogFile("getKey", key);
        return wrappedMap.get(key);
    }

    public int size() {
        return wrappedMap.size();
    }
```

Design Patterns

Wrapper/Decorator

Why not just extend a class?

This is more flexible!

- Accepts any implementation of an interface
- Can be nested with other wrappers (if combined with implementing the interface itself)
- Can limit the methods of the wrapped object (if needed)

```
public class FancyMap<K, V> {

    private final Map<K, V> wrappedMap;

    public FancyMap(Map<K, V> wrappedMap) {
        this.wrappedMap = wrappedMap;
    }

    public V put(K key, V value) {
        prettyPrintToLogFile("putKey", key);
        prettyPrintToLogFile("putValue", value);
        return wrappedMap.put(key, value);
    }

    public V get(K key) {
        prettyPrintToLogFile("getKey", key);
        return wrappedMap.get(key);
    }

    public int size() {
        return wrappedMap.size();
    }
```

Design Patterns

Wrapper/Decorator Xtreme: Proxy

Hidden by the classloader

Doesn't change the type of the object

```
public class DatabaseProxy {

    public static Database getProxiedDatabase(Database wrapped) {
        return (Database) Proxy.newProxyInstance(
                DatabaseProxy.class.getClassLoader(),
                new Class[] {Database.class}, new InvocationHandler() {
            @Override
            public Object invoke(Object proxy, Method method, Object[] args)
                    throws Throwable {

                prettyPrintToLogFile("calling" + method.getName(), Arrays.toString(args));
                return method.invoke(wrapped, args);

            }
        });
    }
```

Advantage: automatically updates as the class/interface changes (reduces maintenance burden).

Often used for security checks or logging frameworks

Your Turn!

1. Use the starter code in the exercise2 package
2. Use the Wrapper pattern to create an Animal implementation that stops you from trying to pet an unfriendly animal
3. Use the wrapper you created to add some safety precautions to the PettingZoo - when you’re done, the PettingZooTest should pass

Design Patterns

Visitor

Problem: I have a collection of related objects of different types, and I want users to be able to interact with the collection seamlessly

Normally, I'd use an **iterator**, but the types are all wrong!

How can I make 'choose how to interact with an object' part of my API?

```
public class Widget<K> {

    private final Map<K, List<String>> stringData
        = new HashMap<>();
    private final Map<K, List<Integer>> intData
        = new HashMap<>();
```

Design Patterns

Visitor

Solution: Give users a structured way to specify per-type behavior

- If types change over time, much easier to find and fix problems vs an iterator with instanceof/type casts
- Easier to implement as a user (what types are present is enforced by compiler)

```
public class Widget<K> {

    private final Map<K, List<String>> stringData
        = new HashMap<>();
    private final Map<K, List<Integer>> intData
        = new HashMap<>();

    public static interface DataVisitor {
        void visitString(String s);
        void visitInt(int i);
    }

    public void visitData(K key, DataVisitor visitor) {
        for (String s : stringData.get(key)) {
            visitor.visitString(s);
        }
        for (int i : intData.get(key)) {
            visitor.visitInt(i);
        }
    }
```

Design Patterns

Visitor

Solution: Give users a structured way to specify per-type behavior

- If types change over time, much easier to find and fix problems vs an iterator with instanceof/type casts
- Easier to implement as a user (what types are present is enforced by compiler)

```
public static void doAThing(Widget<String> widget) {
    widget.visitData("userInput", new DataVisitor() {

        @Override
        public void visitString(String s) {
            displayOutput(s);
        }

        @Override
        public void visitInt(int i) {
            promptUserForInput(i);
        }
    });
}
```

Design Patterns

Visitor

Additional use: structured type casting

If you have an interface where you need a compile-time check of the types of things that implement it (say, hypothetically, an InputConfiguration), visitors let you enforce that

```
public interface Fruit {
    public static interface FruitVisitor {
        void visitApple(Apple apple);
        void visitOrange(Orange orange);
    }

    public static void visitFruit(Fruit f, FruitVisitor visitor) {
        if (f instanceof Apple) {
            visitor.visitApple((Apple)f);
        }
        if (f instanceof Orange) {
            visitor.visitOrange((Orange)f);
        }
    }
}
```

Design Patterns

Visitor

Additional use: structured type casting

If you have an interface where you need a compile-time check of the types of things that implement it (say, hypothetically, an InputConfiguration), visitors let you enforce that

```
public void eatAFruit(Bowl bowl) {
    Fruit f = bowl.grabFruit();
    if (f instanceof Orange) {
        eat(((Orange)f).removePeel());
    }
    if (f instanceof Apple) {
        eat((Apple)f);
    }
}
```

```
public void eatAFruit(Bowl bowl) {
    Fruit.visitFruit(bowl.grabFruit(), new FruitVisitor() {

        @Override
        public void visitOrange(Orange orange) {
            eat(orange.removePeel());
        }

        @Override
        public void visitApple(Apple apple) {
            eat(apple);
        }
    });
}
```

Design Patterns

Visitor

Additional use: structured type casting

If you have an interface where you need a compile-time check of the types of things that implement it (say, hypothetically, an InputConfiguration), visitors let you enforce that

```
public void throwAnApple(Bowl bowl) {
    Fruit.visitFruit(bowl.grabFruit(), new FruitVisitor() {

        @Override
        public void visitOrange(Orange orange) {
            // oranges are cool
        }

        @Override
        public void visitApple(Apple apple) {
            throw new RuntimeException("Keep a doctor away!");
        }
    });
}

public void eatAFruit(Bowl bowl) {
    Fruit.visitFruit(bowl.grabFruit(), new FruitVisitor() {

        @Override
        public void visitOrange(Orange orange) {
            eat(orange.removePeel());
        }

        @Override
        public void visitApple(Apple apple) {
            eat(apple);
        }
    });
}
```

Design Patterns

Visitor

What happens if we add Bananas?

- Update FruitVisitor to have visitBanana()

Option 1: Include a default no-op Banana behavior (backwards-compatible)

Option 2: Compiler will find you every place that needs to be updated (guaranteed)

```
public interface Fruit {
    public static interface FruitVisitor {
        void visitApple(Apple apple);
        void visitOrange(Orange orange);
        default void visitBanana(Banana b) { /* do nothing */ };
    }

    public static void visitFruit(Fruit f, FruitVisitor visitor) {
        if (f instanceof Apple) {
            visitor.visitApple((Apple)f);
        }
        if (f instanceof Orange) {
            visitor.visitOrange((Orange)f);
        }
        if (f instanceof Banana) {
            visitor.visitBanana((Banana) f);
        }
    }
}
```

Design Patterns

Visitor (Alternate)

Is there any way to get rid of those instanceof checks entirely?

⇒ Have the fruit itself handle the type checking

But:

- the downside in terms of readability means in practice this is not a good option (breaks OOP best practices with inversion of control)
- still have to exhaustively list all types in the Visitor definition

⇒ Good to recognize, not to use

Best practices constantly evolve depending on what works **in practice** - this was a good idea that didn't turn out to work well

```
public interface Fruit {
    public void accept(FruitVisitor visitor);

    public static interface FruitVisitor {
        void visitApple(Apple apple);
        void visitOrange(Orange orange);
    }
}

public class Apple implements Fruit {
    @Override
    public void accept(Fruit.FruitVisitor visitor) {
        visitor.visitApple(this);
    }
}
```

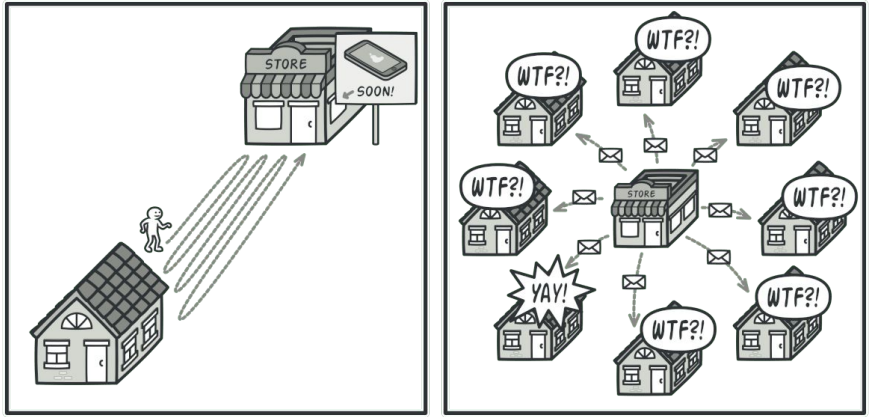
```
public void eatAFruit(Bowl bowl) {
    bowl.grabFruit().accept(new FruitVisitor() {
        @Override
        public void visitApple(Apple apple) {
            eat(apple);
        }
        @Override
        public void visitOrange(Orange orange) {
            eat(orange.removePeel());
        }
    });
```

Design Patterns

Listener/Subscriber

Problem: When some data changes, I want to tell any other pieces of code that might be interested, but I don't know what they all are up front.

How can I introduce runtime connections between components tied to changing data?



Design Patterns

Listener/Subscriber

Solution: A component can **subscribe** to a particular set of changes

```
public class DataModel {

    public static interface DataListener {
        void dataChanged(String key, String oldValue, String newValue);
    }

    private final List<DataListener> listeners = new ArrayList<>();
    private final Map<String, String> data = new HashMap<>();

    public void subscribe(DataListener listener) {
        listeners.add(listener);
    }

    public void updateData(String key, String value) {
        String oldValue = data.put(key, value);
        for (DataListener listener : listeners) {
            listener.dataChanged(key, oldValue, value);
        }
    }
```

```
public void addModalListener(DataModel model) {
    DataListener modalListener = (key, oldValue, newValue) -> {
        if (oldValue != null && !oldValue.isBlank()) {
            displayModal("Data for " + key +
                    " has changed from " + oldValue +
                    " to " + newValue);
        }
    };
    model.subscribe(modalListener);
}
```

Design Patterns

Listener/Subscriber

When to use listener/subscriber vs. having 'on data changed' logic in the data class:

- Generic data classes
- Many different behaviors for listeners

```
public class DataModel {

    public static interface DataListener {
        void dataChanged(String key, String oldValue, String newValue);
    }

    private final List<DataListener> listeners = new ArrayList<>();
    private final Map<String, String> data = new HashMap<>();

    public void subscribe(DataListener listener) {
        listeners.add(listener);
    }

    public void updateData(String key, String value) {
        String oldValue = data.put(key, value);
        for (DataListener listener : listeners) {
            listener.dataChanged(key, oldValue, value);
        }
    }
```

```
public void addModalListener(DataModel model) {
    DataListener modalListener = (key, oldValue, newValue) -> {
        if (oldValue != null && !oldValue.isBlank()) {
            displayModal("Data for " + key +
                    " has changed from " + oldValue +
                    " to " + newValue);
        }
    };
    model.subscribe(modalListener);
}
```

Your Turn!

1. Start with the code in the exercise3 package
2. Use the Listener pattern to fix the TODOs in StudentDirectory, CSDepartment, and ITDepartment

Coding Concept Patterns

Somewhere between a "best practice" and a "design pattern"

Still matches a problem to a solution, but in a more abstract way than design patterns

Resilient Code

Goes by many names (code path/execution isolation, defensive coding, etc)

Goal: If Component A has a bug/becomes malicious, Component B should still work

Solutions:

- Sanitize/validate inputs for **public** methods and constructors
- Isolate method calls by default (clients must pass a key or some other info to connect different calls)
- Methods can be called in any order (hide implementation details behind wrapper methods)

Resilient Code

Good:

```
public class Widget {

    private final Database database;
    private boolean initialized = false;

    public Widget(Database database) {
        this.database = database;
    }

    public void doAThing() {
        if (!initialized) {
            database.initializeSchema();
            initialized = true;
        }
        actuallyDoAThing();
    }

    private void actuallyDoAThing() {
        // TODO
    }
}
```

Fragile:

```
public class Widget {

    private final Database database;

    public Widget(Database database) {
        this.database = database;
    }

    /**
     * Make sure to call this before using the Widget,
     * but only call it once.
     */
    public void initialize() {
        database.initializeSchema();
    }

    public void doAThing() {
        actuallyDoAThing();
    }

    private void actuallyDoAThing() {
        // TODO
    }
}
```

Resilient Code

Good:

```
public class Widget {

    private Map<UUID, Integer> intermediateValues = new HashMap<>();

    public UUID doStepOne(int val) {
        int intermediateValue = val*2;
        UUID key = UUID.randomUUID();
        intermediateValues.put(key, intermediateValue);
        return key;
    }

    public int doStepTwo(UUID stepOneKey) {
        return intermediateValues.get(stepOneKey) + 1;
    }
}
```

Fragile:

```
public class Widget {

    private int intermediateValue;

    public void doStepOne(int val) {
        intermediateValue = val*2;
    }

    public int doStepTwo() {
        return intermediateValue + 1;
    }
}
```

Stateless Services

Goal: Prevent multiple callers of a method/clients of a server from messing with each other

Solution:

Move state management from implementation to client

Stateless Services

Adding multiple clients is risky/difficult:

```
public static interface Server {
      public void initializeWorkflow(String startingData);
      public void runWorkflow();
      public String getResultsOfWorkflow();
}

public void client(Server server) {
      server.initializeWorkflow("Hello World");
      server.runWorkflow();
      String result = server.getResultsOfWorkflow();
}
```

⇒ Use parameters/return values to track **client-related** state; implementation should only store client-independent state

Adding multiple clients is easy:

```
public static interface Server {
  public String runWorkflow(String startingData);
}

public void client(Server server) {
  String result = server.runWorkflow("Hello World");
}
```

Streaming Inputs/Outputs

Large (or potentially large) inputs

- Doesn't fit in memory
- Fits in memory, but it's slow
- Usually fits in memory, but sometimes…

Inputs without a known-beforehand end point

- Files
- Network socket inputs
- Running a 'where' query on a database table

Streaming Solutions

Iterable<T>/Iterator<T>

- Most **general**/flexible
- Iterable is nicer than iterator (can be used in for loops), but requires that the input can be **restarted**
- Typically used when the **entire** stream needs to be read

Streaming Solutions

Stream<T>

- Introduced in Java 8
- Rarely explicitly instantiated
- Good for **pipelining** operations
- Can become unreadable quickly, be careful about making your pipeline too dense!
- **map** means 'transform'

Streaming Solutions

Good example of how lambdas can become too dense - remember to turn lambdas into their **own named variables** when there is more than 1

What even is this:

```
new ArrayList<String>().stream()
    .filter(s -> s.indexOf(" ") == -1)
    .map(s -> s.toUpperCase())
    .distinct()
    .flatMapToInt(s -> s.chars());
```

Better:

```
new ArrayList<String>().stream()
    .filter(removeStringsWithSpaces)
    .map(toUpperCase)
    .distinct()
    .flatMapToInt(addAsciiValueOfCharacters);
```

Streaming Inputs/Outputs

I HATE FEELING DESPERATE ENOUGH TO VISIT THE SECOND PAGE OF GOOGLE RESULTS.

Paging: define a page size and a hasMore flag

- Good for situations where only **some** of the input is needed
- Choose the page size to fit nicely in memory

  (typically a List<T>)

- Allows **batching** of downstream processing (ex: printing)
- Can include the number of pages, or (more common) an **approximate** number of pages
- Tricky bit: getNextPage
  - Deterministic ordering, $O(n^2)$ traversal
  - Iterator pointer (can get into memory leak situations)

Your Turn!

1. Start with the code in the exercise4 package
2. Using an Iterator, change findMaximumValue to accept an arbitrarily large number of input integers

Let's Review!

Code Conventions: Compact iteration, lambdas, generics (syntax)

Design Patterns: Problems/Basic solutions for Builders, Dependency Injection, Lazy Initialization, Wrappers, Visitors, Listeners

Resilient Code: 3 ways to make a component robust to failure in other components

Streaming: When to use Collection vs Iterable vs Iterator vs Paging